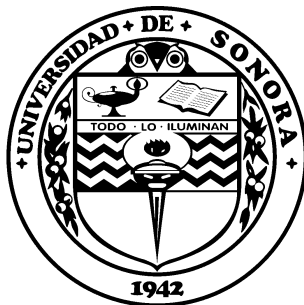




Universidad de Sonora

Licenciatura en Ciencias de la Computación

Resumen del Capítulo 4: Hilos (Threads)

Operating System Concepts Essentials, 2^a edición
Silberschatz, Galvin y Gagne

Materia: Sistemas Operativos

Periodo: 2026-2

Profesor: Dr. Donald José Rodríguez Úbeda

Integrantes del equipo

Angel Alberto Amaya Soria	Expediente 223210350
Angel Fernando Bórquez Guerrero	Expediente 219208106
Javier Leonardo Miranda Sánchez	Expediente 223215039
Oliver Ruiz Beltrán	Expediente 223204115

Hermosillo, Sonora

Septiembre de 2026

1. Introducción

En el capítulo anterior del libro, el modelo de proceso se había presentado como un programa en ejecución con un único flujo de control. Sin embargo, prácticamente todos los sistemas operativos modernos permiten que un proceso contenga varios flujos de control simultáneos, conocidos como hilos. Este capítulo explica qué es un hilo, cuáles son sus beneficios frente al uso exclusivo de procesos, cómo se relacionan los hilos de usuario con los hilos del kernel, las principales bibliotecas para crearlos (Pthreads, Windows y Java), distintas estrategias de creación implícita de hilos, los problemas de diseño que introduce la programación multihilo y, finalmente, cómo Windows y Linux implementan el soporte de hilos a nivel de kernel.

2. El concepto de hilo y su motivación

Un hilo es la unidad básica de utilización del CPU: se compone de un identificador, un contador de programa, un conjunto de registros y una pila propia. A diferencia de estos elementos, que son exclusivos de cada hilo, la sección de código, la sección de datos y otros recursos del sistema operativo (como los archivos abiertos) se comparten entre todos los hilos que pertenecen al mismo proceso. Un proceso tradicional, de un solo hilo, únicamente puede realizar una tarea a la vez; en cambio, un proceso con múltiples hilos puede ejecutar varias tareas de forma concurrente.

La mayoría del software actual está diseñado para trabajar con varios hilos. Un navegador, por ejemplo, puede usar un hilo para mostrar el contenido de una página y otro para descargar datos de la red; un procesador de textos puede dedicar un hilo a la interfaz gráfica, otro a capturar las teclas presionadas por el usuario y un tercero a revisar la ortografía en segundo plano. Los servidores también se benefician enormemente de los hilos: en lugar de crear un proceso completo para atender cada solicitud de un cliente una operación costosa en tiempo y recursos, resulta más eficiente contar con un único proceso que despache un hilo nuevo por cada solicitud, mientras otro hilo continúa escuchando peticiones adicionales. De forma similar, las llamadas a procedimiento remoto (RPC) suelen atenderse mediante hilos, y los propios núcleos de los sistemas operativos actuales son multihilo, dedicando hilos específicos a tareas como el manejo de interrupciones o la administración de memoria libre.

Los beneficios de la programación multihilo pueden resumirse en cuatro categorías: mayor **capacidad de respuesta**, ya que una operación larga puede ejecutarse en un hilo distinto sin bloquear la interacción con el usuario; **compartición de recursos**, pues los hilos de un mismo proceso comparten memoria y recursos de forma automática, sin necesidad de mecanismos explícitos como los que requieren los procesos independientes; **economía**, dado que crear y cambiar de contexto entre hilos es considerablemente más barato que hacerlo entre procesos (en Solaris, por ejemplo, crear un proceso es unas treinta veces más lento que crear un hilo); y **escalabilidad**, ya que en arquitecturas con varios procesadores los hilos de un mismo proceso pueden ejecutarse verdaderamente en paralelo, cosa que un proceso de un solo hilo no puede aprovechar sin importar cuántos núcleos existan.

3. Programación en sistemas multinúcleo

Así como los sistemas de un solo procesador evolucionaron hacia sistemas de múltiples procesadores, la tendencia actual es colocar varios núcleos de cómputo dentro de un mismo chip. La programación multihilo aprovecha estas arquitecturas de forma natural: en un sistema de un solo núcleo, los hilos únicamente pueden intercalarse en el tiempo, mientras que en un sistema de varios núcleos, distintos hilos pueden asignarse a distintos núcleos y ejecutarse verdaderamente en paralelo.

Resulta importante distinguir entre *paralelismo* y *conurrencia*: un sistema es paralelo cuando puede realizar más de una tarea al mismo tiempo, mientras que un sistema es concurrente cuando soporta más de una tarea permitiendo que todas avancen, aunque no se ejecuten simultáneamente. Por lo tanto, es posible tener conurrencia sin paralelismo, como ocurría en los sistemas de un solo procesador antes de la llegada de las arquitecturas multinúcleo.

El diseño de aplicaciones para sistemas multinúcleo enfrenta cinco retos principales: identificar las tareas que pueden dividirse y ejecutarse de forma independiente; balancear la carga de trabajo entre ellas, de modo que dedicar un núcleo completo a una tarea sea realmente conveniente; dividir los datos que cada tarea utilizará; gestionar las dependencias de datos entre tareas, sincronizándolas cuando una depende del resultado de otra; y, finalmente, enfrentar la mayor dificultad para probar y depurar programas concurrentes, dado que existen muchas más rutas de ejecución posibles que en un programa de un solo hilo.

Existen dos tipos de paralelismo. El **paralelismo de datos** distribuye subconjuntos de un mismo conjunto de datos entre varios núcleos, realizando la misma operación sobre cada subconjunto (por ejemplo, sumar la mitad de un arreglo en cada núcleo). El **paralelismo de tareas** distribuye, en cambio, tareas distintas entre los núcleos, de modo que cada hilo realiza una operación diferente, ya sea sobre los mismos datos o sobre datos distintos. En la práctica, la mayoría de las aplicaciones combinan ambos enfoques.

4. Modelos de multihilo

El soporte de hilos puede ofrecerse a nivel de usuario o a nivel de kernel. Los hilos de usuario se administran por completo en espacio de usuario, sin intervención del núcleo, mientras que los hilos de kernel son gestionados directamente por el sistema operativo. Dado que en algún punto debe existir una relación entre ambos tipos, se han propuesto tres modelos principales.

En el **modelo muchos a uno**, varios hilos de usuario se asignan a un único hilo de kernel; la administración es eficiente porque ocurre en espacio de usuario, pero si un hilo realiza una llamada al sistema que lo bloquea, todo el proceso se bloquea, y además ningún hilo puede aprovechar varios núcleos a la vez. Este modelo, usado por ejemplo en la biblioteca de hilos verdes de Solaris y en las primeras versiones de Java, ha caído en desuso.

En el **modelo uno a uno**, cada hilo de usuario se asocia con su propio hilo de kernel, lo

cual permite mayor concurrencia (otro hilo puede ejecutarse si uno se bloquea) y verdadero paralelismo en sistemas multiprocesador; su desventaja es que crear cada hilo de usuario implica crear también un hilo de kernel, por lo que suele limitarse el número total de hilos permitidos. Linux y la familia de sistemas Windows utilizan este modelo.

El **modelo muchos a muchos** multiplexa un número de hilos de usuario sobre un número menor o igual de hilos de kernel, combinando las ventajas de los dos modelos anteriores: el programador puede crear tantos hilos de usuario como necesite, y el núcleo puede ejecutar varios de ellos en paralelo o continuar con otro hilo cuando uno se bloquea. Una variante de este modelo, conocida como **modelo de dos niveles**, permite además enlazar un hilo de usuario específico directamente a un hilo de kernel; este esquema fue utilizado por Solaris en versiones anteriores a la 9, a partir de la cual se adoptó el modelo uno a uno.

5. Bibliotecas de hilos

Una biblioteca de hilos ofrece al programador una interfaz (API) para crear y administrar hilos, ya sea implementada completamente en espacio de usuario o como una biblioteca de kernel respaldada por el sistema operativo. Las tres bibliotecas más utilizadas actualmente son Pthreads, Windows y Java. **Pthreads** es la extensión de hilos del estándar POSIX (IEEE 1003.1c); se trata de una especificación de comportamiento, no de una implementación concreta, y está disponible principalmente en sistemas de tipo UNIX como Linux, Mac OS X y Solaris. Los **hilos de Windows** conforman una biblioteca de kernel disponible en la familia de sistemas Windows, con una técnica de creación (**CreateThread**) muy similar en espíritu a la de Pthreads. Los **hilos de Java**, por su parte, constituyen el modelo fundamental de ejecución del lenguaje: incluso un programa Java sencillo se ejecuta como un hilo dentro de la máquina virtual (JVM); dado que ésta corre sobre un sistema operativo anfitrión, en la práctica los hilos de Java terminan apoyándose en la biblioteca nativa de ese sistema (la API de Windows o Pthreads, según el caso).

En Pthreads y Windows, los datos declarados de forma global se comparten automáticamente entre todos los hilos del proceso; en Java, al no existir el concepto de datos globales, la compartición debe organizarse explícitamente pasando referencias a un objeto común entre los hilos interesados. En los tres casos, el patrón más común para crear hilos es el de *fork-join* (o hilos síncronos): el hilo padre crea uno o varios hilos hijos y espera a que todos terminen antes de continuar, generalmente para combinar los resultados que cada uno calculó; esto contrasta con los hilos asíncronos, en los que el hilo padre no necesita esperar a que sus hilos hijos terminen, como ocurre en el ejemplo del servidor multihilo descrito anteriormente.

6. Hilos implícitos

Ante el crecimiento en el número de núcleos disponibles, resulta cada vez más difícil que el programador diseñe manualmente aplicaciones con cientos o miles de hilos. Por ello ha surgido la estrategia de **hilos implícitos**, en la que la creación y administración de los hilos se delega a los compiladores y a las bibliotecas de tiempo de ejecución.

Los **grupos de hilos** (*thread pools*) resuelven dos problemas del enfoque de crear un hilo nuevo por cada solicitud: el tiempo que toma crear y destruir cada hilo, y el riesgo de agotar los recursos del sistema si no existe un límite al número de hilos activos. La idea consiste en crear un conjunto fijo de hilos al iniciar el proceso, que permanecen a la espera de trabajo; cuando llega una solicitud, se despierta un hilo disponible del grupo, y al terminar su tarea el hilo regresa al grupo en lugar de destruirse.

OpenMP es un conjunto de directivas de compilador y una API para programas en C, C++ o FORTRAN, pensada para la programación paralela en entornos de memoria compartida. El programador simplemente marca con directivas (como `#pragma omp parallel`) las regiones de código que pueden ejecutarse en paralelo, y OpenMP crea automáticamente tantos hilos como núcleos de procesamiento existan en el sistema, incluyendo la posibilidad de repartir automáticamente las iteraciones de un ciclo `for` entre ellos.

Grand Central Dispatch (GCD) es la tecnología de Apple para macOS e iOS que, de forma similar a OpenMP, se encarga de administrar la mayor parte de los detalles del uso de hilos. Introduce el concepto de *bloque*, una unidad de trabajo autocontenida, que se coloca en una cola de despacho para su ejecución. GCD distingue entre colas seriales, donde los bloques se ejecutan uno tras otro en orden FIFO, y colas concurrentes, donde varios bloques pueden extraerse y ejecutarse en paralelo según su prioridad (baja, por defecto o alta). Internamente, el grupo de hilos que utiliza GCD está compuesto por hilos POSIX, cuyo número el propio sistema ajusta dinámicamente según la demanda.

Además de estas tres estrategias, existen otras alternativas comerciales, como las Threading Building Blocks de Intel o el paquete `java.util.concurrent` de Java, orientado a facilitar la creación y gestión implícita de hilos.

7. Cuestiones relevantes al diseñar programas multihilo

El diseño de programas multihilo introduce varios problemas que no existen o que se resuelven de forma distinta en programas de un solo hilo.

Las llamadas `fork()` y `exec()` cambian de sentido en un programa multihilo: cuando un hilo invoca `fork()`, cabe preguntarse si el nuevo proceso debe duplicar todos los hilos existentes o únicamente el hilo que hizo la llamada; algunos sistemas UNIX ofrecen ambas variantes. En cambio, `exec()` siempre reemplaza el proceso completo, incluidos todos sus hilos, por lo que si se planea invocar `exec()` inmediatamente después de `fork()`, basta con duplicar el hilo que hizo la llamada.

El manejo de señales también se complica en presencia de varios hilos: una señal síncrona, generada por la propia ejecución del hilo (como un acceso ilegal a memoria), debe entregarse al hilo que la provocó; pero para las señales asíncronas, generadas por un evento externo, existen varias opciones entregarla a un hilo específico, a todos los hilos, a un subconjunto de ellos, o designar un hilo encargado de recibir todas las señales del proceso. Windows no ofrece señales de forma nativa, pero permite emular su comportamiento mediante llamadas a procedimiento asíncronas (APC), que a diferencia de las señales de UNIX

se entregan directamente a un hilo en particular.

La cancelación de hilos consiste en terminar un hilo antes de que complete su trabajo, como ocurre cuando varios hilos buscan en paralelo dentro de una base de datos y uno de ellos ya encontró el resultado buscado. Puede realizarse de forma asíncrona, terminando el hilo objetivo de inmediato, o de forma diferida, en la que el propio hilo revisa periódicamente si existe una solicitud de cancelación pendiente y se termina a sí mismo en un punto seguro; esta segunda variante es la recomendada, ya que la cancelación asíncrona puede dejar recursos compartidos en un estado inconsistente.

El almacenamiento local al hilo (TLS) permite que cada hilo mantenga su propia copia de ciertos datos, a diferencia de las variables locales de una función, que solo existen mientras dura esa invocación; resulta útil, por ejemplo, para asignar un identificador único a cada transacción que atiende un hilo distinto.

Finalmente, **las activaciones del planificador** son un mecanismo de comunicación entre la biblioteca de hilos de usuario y el núcleo, necesario en los modelos muchos a muchos y de dos niveles. El núcleo asigna a la aplicación un conjunto de procesadores virtuales conocidos como procesos ligeros (LWP), y cuando un hilo está a punto de bloquearse, el núcleo realiza una *upcall* hacia la biblioteca para que ésta reasigne otro hilo al procesador virtual liberado, manteniendo así el nivel de paralelismo deseado.

8. Soporte de hilos en Windows y Linux

Windows implementa el modelo uno a uno, de modo que cada hilo de usuario tiene asociado un hilo de kernel. Cada hilo se compone de un identificador, un conjunto de registros, una pila de usuario y otra de kernel, y un área de almacenamiento privada para bibliotecas en tiempo de ejecución. Sus estructuras de datos principales son el bloque de hilo ejecutivo (ETHREAD) y el bloque de hilo de kernel (KTHREAD), que residen exclusivamente en espacio de kernel, y el bloque de entorno de hilo (TEB), una estructura en espacio de usuario que contiene, entre otros datos, el identificador del hilo y el área de almacenamiento local al hilo.

Linux, en cambio, no distingue conceptualmente entre procesos e hilos: ambos se representan mediante una *tarea* (*task*), y ofrece la llamada al sistema `clone()` para crear una nueva tarea especificando, mediante un conjunto de banderas, qué recursos compartirá con la tarea que la originó (información del sistema de archivos, espacio de memoria, manejadores de señales o archivos abiertos). Si se activan todas estas banderas, el resultado es equivalente a crear un hilo tradicional; si no se activa ninguna, el comportamiento es equivalente al de la llamada clásica `fork()`. Esta flexibilidad es posible porque cada tarea del kernel de Linux se representa mediante una estructura (`struct task_struct`) que no almacena los datos directamente, sino que apunta a las estructuras compartidas o copiadas según el caso.

9. Conclusión

Un hilo es, en esencia, un flujo de control independiente dentro de un proceso, y un proceso multihilo puede mantener varios de estos flujos dentro del mismo espacio de direcciones. El uso de hilos aporta beneficios claros en capacidad de respuesta, compartición de recursos, economía y escalabilidad, especialmente en la era de los sistemas multinúcleo. La relación entre los hilos que ve el programador y los que administra el núcleo puede establecerse mediante distintos modelos (muchos a uno, uno a uno o muchos a muchos), y existen tanto bibliotecas explícitas (Pthreads, Windows, Java) como estrategias de hilos implícitos (grupos de hilos, OpenMP, Grand Central Dispatch) para facilitar su uso. No obstante, programar con hilos introduce retos particulares el comportamiento de `fork()` y `exec()`, el manejo de señales, la cancelación de hilos, el almacenamiento local y la coordinación entre la biblioteca de hilos y el núcleo que todo diseñador de sistemas operativos y todo programador de aplicaciones concurrentes debe comprender.

Referencias bibliográficas

Silberschatz, A., Galvin, P. B., y Gagne, G. (2014). *Operating system concepts essentials* (2.^a ed.), Capítulo 4: Threads. John Wiley & Sons.